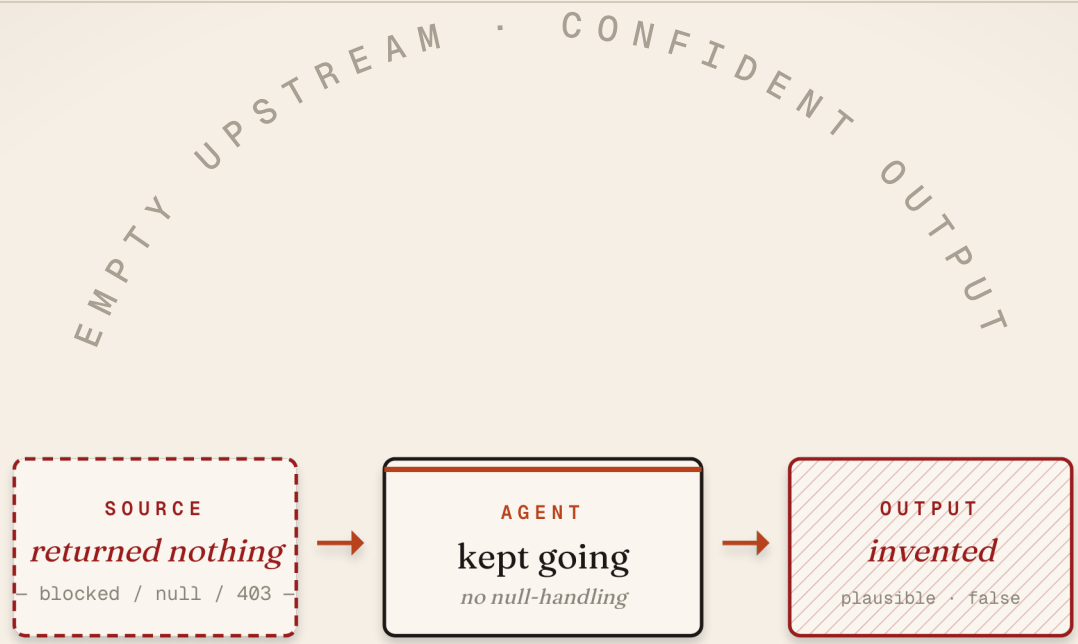

FIELD NOTES · NO. 02 · A GENERATIVE AI FIELD GUIDE

I shipped an AI agent that *invented* its own job candidates.

Three nodes. Two hours. The failure mode every production agent eventually meets — and the four defences that ship them anyway.

— THE FUNDAMENTAL

> *When the upstream source returns nothing, an LLM agent fills the gap. With anything plausible.*

This is not a bug. It is the **default behaviour** of any agent that wraps an LLM and does not explicitly handle empty input. Production agents fail here more than anywhere else.

— THE FAILURE MODE BEHIND MOST AGENT REGRETS

— THE MECHANISM

LLMs complete *patterns.* Empty input is still a pattern.

01	Your prompt sets up a ***shape*** the model has seen before.	SETUP
02	The upstream tool / API / source returns *null, blocked, or 403*.	TRIGGER
03	Without explicit "stop here" guardrails, the model *completes the pattern anyway* — from training data, from priors, from anything that fits.	HALLUCINATE
04	Output looks correct in shape, in tone, in structure. *It isn't.*	SHIP IT

The model isn't lying. It's doing what it was trained to do — **complete the next token**. The bug is in the pipeline around it.

— A REAL CASE

We built a 3-node HR screener. *Same failure, exact shape.*

— THE PIPELINE · NO CODE · THREE LLM NODES WIRED IN SEQUENCE —

NODE 01
Profile Search

→

NODE 02
Skill Extractor

→

NODE 03
Candidate Scorer

Inputs **location · years · job title** → *Node 1 searches the web for 5 profiles → Node 2 extracts the top 5 skills for the title → Node 3 scores the 5 profiles against the 5 skills and returns a* **ranked shortlist**.

On paper, clean. In production, Node 1 hit a wall — and the agent obeyed the fundamental from slide 2.

— THE RECEIPT

Node 1 returned *nothing.* Node 3 ranked them *anyway.*

– WHAT WE EXPECTED –

5 real profiles · verifiable URLs

linkedin.com/in/ananya-kumar-eng

linkedin.com/in/raj-vora-react

linkedin.com/in/sneha-iyer-fe

linkedin.com/in/karthik-bn-ux

linkedin.com/in/priya-rao-web

Real people. Real public profiles. Real data.

– WHAT WE GOT –

5 fabricated profiles · all plausible · all false

~~linkedin.com/in/alex-frontend-2024~~

~~linkedin.com/in/sarah-react-blr~~

~~linkedin.com/in/m-johnson-sde~~

~~linkedin.com/in/dev-banerjee-fe~~

~~linkedin.com/in/p-sharma-react~~

None exist. Plausibility is not truth.

The pipeline produced a *ranked shortlist* — of people who don't exist. A senior recruiter, given this output, would have wasted a week.

5/10

— GENERALISE

Your data is almost always *somebody else's protected asset.*

in	LinkedIn *profiles, jobs, posts*	BLOCKED
𝕏	Twitter / X *tweets, users*	PAID API
☐	Indeed / Naukri *job postings*	TOS-BLOCKED
⌥	Glassdoor *reviews*	TOS-BLOCKED
⚑	GitHub *public profiles, repos*	OPEN API
◉	Reddit *posts, comments*	PAID API

The web you remember was scrapable. The web that matters in 2026 isn't. *Design for that before you build.*

— DEFENCE 01 · NULL-HANDLING

01/04

Make "*return empty*" an explicit option.

– ADD TO EVERY NODE THAT CALLS AN EXTERNAL SOURCE –

```
If the source returns no results, partial results, or an
error, you MUST return exactly: { "status": "empty",
"reason": "<one line>" }.

Do NOT invent URLs, names, dates, or facts to fill the gap.
Do NOT proceed to downstream reasoning on empty input.
```

This single block stops roughly 80% of "the agent made it up" failures. Tested across recruitment, research, and competitive-intel pipelines.

Models are excellent at following explicit refusal instructions. They are terrible at *inferring* when they should refuse.

7/10

02/04

— DEFENCE 02 · CONSTRAIN THE SOURCE

Pick the source *before* you write the prompt.

— REPLACE OPEN "SEARCH THE WEB" WITH A NAMED SOURCE —

WEAK: "Search the web for senior frontend candidates in Bengaluru."

STRONG: "Query **only** the GitHub Public REST API at api.github.com/search/users
for users with location:Bengaluru and language:JavaScript.
Return their public github URL, name, and bio.
If the API rate-limits or returns no users, return { "status": "empty" }."

A named, allowed source removes the model's freedom to "find" data from anywhere — including its imagination.

Open prompts produce open hallucinations. *Constrained prompts produce constrained output* — including honest emptiness.

8/10

03/04

— DEFENCE 03 · A VALIDATOR NODE

Put a *checker* between research and reasoning.

– A 1-NODE CHECK, ADDED BETWEEN NODE 1 AND NODE 2 –

```
For every URL in the input, perform a HEAD request.
If status≠ 200, drop the row.
If fewer than N valid rows remain, return { "status": "insufficient", "found": N }.

Pass through ONLY verified rows to the next node.
```

— DEFENCE 04 · PICK RELIABLE UPSTREAM

Your first three production agents should run on **data you own** or **APIs you control**. Save scraped-web problems for after you've shipped the failure modes once.

— A WORKSHOP THAT FIXES THIS —

Next: *Defensive Prompting* for agents.

Null-handling, validator nodes, constrained sources. Two hours. Build your own agent. No code. Dates in the comments.

DM "SEAT" TO JOIN THE NEXT SESSION

& *Or DM "defences" for the one-page cheat sheet — slides 7–9 in a single printable PDF with copy-ready prompt blocks.*

Shantanu Chandra

Teaching AI fluency to professionals who don't write code. Field notes from live workshops with senior operators.

— CONNECT —

in linkedin.com/in/chandrashantanu